

P1664R5

reconstructible_range - a concept for putting ranges back together

Published Proposal, 2021-08-15

Authors:

[JeanHeyd Meneide](#)

[Hannes Hauswedell](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Target:

C++23

Latest:

https://the-phd.dev/_vendor/future_cxx/papers/d1664.html

Abstract

This paper proposes new concepts to the Standard Library for ranges called Reconstructible Ranges for the purpose of ensuring a range/view broken down into its two iterators can be "glued" back together using an ADL-found function taking a tag, the range's iterator, and the range's sentinel.

Table of Contents

1 Revision History

- 1.1 Revision 5 - August 15th, 2021
- 1.2 Revision 4 - June 15th, 2021
- 1.3 Revision 3 - April 15th, 2021
- 1.4 Revision 2 - January 13th, 2020
- 1.5 Revision 1 - November 24th, 2019
- 1.6 Revision 0 - August, 5th, 2019

2 Motivation

- 2.1 Preservation of Properties
- 2.2 Compile-Time and Interoperability

3 Design

- 3.1 Range? Why not "Borrowed Range"?
- 3.2 Multiple, Safe Reconstruction Points
- 3.3 Opt-in?
- 3.4 Applicability
- 3.5 Why The Overloads?
 - 3.5.1 Unrecoverable Ranges
 - 3.5.1.1 Negotiating
 - 3.5.1.2 Current Design: Lossless Information
 - 3.5.2 Legacy Code
 - 3.5.3 Minimum Viable Customization

4 Deployment Experience

5 Impact

6 Proposed Changes

- 6.1 Feature Test Macro
- 6.2 Intent
- 6.3 Proposed Library Wording
 - 6.3.1 Add a feature test macro `__cpp_lib_reconstructible_range`.
 - 6.3.2 Insert into §24.2 Header `<ranges>` Synopsis [ranges.syn] a new customization point object in the inline namespace:
 - 6.3.3 Insert into §24.4.2 Ranges [range.range]'s after paragraph 7, one additional paragraph:
 - 6.3.4 Insert a new sub-clause "§24.3.13 `ranges::reconstruct` [`range.prim.recons`]", after "§24.3.12 `ranges::cdata` [`range.prim.cdata`]":
 - 6.3.5 Add to "§21.4.3.1 `General`" [string.view.template.general] a `friend` function ADL extension point for `basic_string_view`:
 - 6.3.6 Add a new subclause "§21.4.    `Range Reconstruction`" [string.view.reconstruct] in §21.4:
 - 6.3.7 Add to "§22.7.3.1 `Overview`" [span.overview] a `friend` function ADL extension point for `span`:
 - 6.3.8 Add a new subclause "§22.7.3.    `Range Reconstruction`" [span.reconstruct] in §22.7.3:
 - 6.3.9 Add to "§24.5.4.1 `General`" [range.subrange.general] a `friend` function ADL extension point for `subrange`:
 - 6.3.10 Add a new subclause "§24.5.4.    `Range Reconstruction`" [range.subrange.reconstruct] in §24.5.4:
 - 6.3.11 Add to "§24.6.4.2 Class template `iota_view`" [range.iota] a `friend` function ADL extension point for `iota_view`:
 - 6.3.12 Add one new paragraph to "§24.6.4.2 Class template `iota_view`" [range.iota]:
 - 6.3.13 Add to "§24.6.2.2 Class template `empty_view`" [range.empty.view] a `friend` function ADL extension point for `empty_view`:
 - 6.3.14 Add to "§24.7.6.2 Class template `transform_view`" [range.transform.view] a `friend` function ADL extension point for `transform_view`:

- 6.3.15 Add new paragraphs to "§24.6.4.2 Class template `transform_view`" [range.transform]:
- 6.3.16 Modify "§24.7.7.1 Overview " [range.take.overview] for `views::take`'s paragraph 2:
- 6.3.17 Modify "§24.7.9.1 Overview " [range.drop.overview] for `views::drop`'s paragraph 2:

7 Acknowledgements

References

Informative References

§ 1. Revision History

§ 1.1. Revision 5 - August 15th, 2021

- Vastly improve wording and address e-mail feedback (thanks, Hannes Hauswedell and Barry Revzin).
- Use the `is_enum_v` and `is_class_v` guards to prevent all bad catch-alls (thanks, Christopher Di Bella).
- At the moment, we cannot have the `iterator` and `sentinel` types for things like `iota_view` in the standard-mandated, exposition-only italics, so this little note up here will have to do.
- Ensure that the concepts are properly specified.
- Add lots more motivation to the design, particularly [§ 3.5 Why The Overloads?](#).

§ 1.2. Revision 4 - June 15th, 2021

- Rewrite most of the wording.
- Fix up the design section to always use an ADL Customization Point, all the time. The `is_class_v` and `is_enum_v` protects against most bad cases.

§ 1.3. Revision 3 - April 15th, 2021

- Vastly improve examples.
- Re-target motivation.
- Adjust fixes for things that have changed since C++20.

§ 1.4. Revision 2 - January 13th, 2020

- Improve wording and add more explanation in the design sections for the changes.

§ 1.5. Revision 1 - November 24th, 2019

- Change to `snake_case` concepts. (November 6th)
- Update wording to target [\[n4835\]](#). (November 6th)
- Moved by LWG! 🍷 (November 6th)
- Last minute objection to this paper in plenary.
- Withdrawn; targeting C++23 instead with better design.
- Explored 3 different designs offline: settle on 1 (thanks, Oktal).

§ 1.6. Revision 0 - August, 5th, 2019

- Initial release.

§ 2. Motivation

C++20

```
template <typename T>
using span = quickcpplib::span<T>;

std::vector<int> vec{1, 2, 3, 4, 5};
span<int> s{vec.data(), 5};
span<int> v = s | views::drop(1) | views::take(10)
              | views::drop(1) | views::take(10);
```

✗ - Compilation error

```
template <typename T>
using span = quickcpplib::span<T>;

std::vector vec{1, 2, 3, 4, 5};
span<int> s{vec.data(), 5};
auto v = s | views::drop(1) | views::take(10)
           | views::drop(1) | views::take(10);
```

⚠ - Compiles, but `decltype(v)` is `ranges::take_view<ranges::drop_view<ranges::take_view<ranges::drop_view<span<int>, dynamic_extent>>>>>`

```
template <typename T>
using span = quickcpplib::span<T>;

std::vector<int> vec{1, 2, 3, 4, 5};
span<int> s{vec.data(), 5};
span<int> v = s | views::drop(1) | views::take(10)
              | views::drop(1) | views::take(10);
```

✓ - Compiles and works with 1

```
template <typename T>
using span = quickcpplib::span<T>;

std::vector vec{1, 2, 3, 4, 5};
span<int> s{vec.data(), 5};
auto v = s | views::drop(1) | views::take(10)
           | views::drop(1) | views::take(10);
```

✓ - Compiles and works with 1
quickcpplib::span<int>

```
std::u8string name = "אֵלֶּםִּי־מִתְעַלֵּם גַּן־לְעֹלָם־לְעֹלָם";
char16_t conversion_buffer[432];

std::u8string_view input(name);
std::span<char16_t> output(conversion_buffer, 432);

auto encoding_result = ztd::text::transcode(input, output);
// compiler error here !!
std::u8string_view unprocessed_code_units
    = encoding_result.input;
std::span<char16_t> unconsumed_output
    = encoding_result.output;
```

✗ - Compilation error

```
std::u8string name = "אֵלֶּםִּי־מִתְעַלֵּם גַּן־לְעֹלָם־לְעֹלָם";
char16_t conversion_buffer[432];

std::u8string_view input(name);
std::span<char16_t> output(conversion_buffer, 432);

auto encoding_result = ztd::text::transcode(input, output);
auto unprocessed_code_units = encoding_result.input;
auto unconsumed_output = encoding_result.output;
```

⚠ - Compiles, but `decltype(unprocessed_code_units)` is `ranges::subrange<std::u8string_view::iterator, std::u8string_view::iterator>` and `decltype(unconsumed_output)` is `ranges::subrange<std::span<char16_t, std::dynamic_extent>::iterator, std::span<char16_t, std::dynamic_extent>::iterator>`

```
std::u8string name = "אֵלֶּםִּי־מִתְעַלֵּם גַּן־לְעֹלָם־לְעֹלָם";
char16_t conversion_buffer[432];

std::u8string_view input(name);
std::span<char16_t> output(conversion_buffer, 432);

auto encoding_result = ztd::text::transcode(input, output);
std::u8string_view unprocessed_code_units
    = encoding_result.input;
std::span<char16_t> unconsumed_output
    = encoding_result.output;
```

✓ - Compiles and works, types

```
std::u8string name = "אֵלֶּםִּי־מִתְעַלֵּם גַּן־לְעֹלָם־לְעֹלָם";
char16_t conversion_buffer[432];

std::u8string_view name(input.data(), input.size());
std::span<char16_t> output(conversion_buffer, 432);

auto encoding_result = ztd::text::transcode(name, output);
auto unprocessed_code_units = encoding_result.input;
auto unconsumed_output = encoding_result.output;
```

✓ - Compiles and works, where `std::u8string_view` and `std::span<char16_t>`.

Currently in C++, there is no Generic ("with a capital G") way to take a range apart with its iterators and put it back together. That is, the following code is not guaranteed to work:

```
template <typename Range>
auto operate_on_and_return_updated_range (Range&& range) {
    using I = std::ranges::iterator_t<Range>;
    using S = std::ranges::sentinel_t<Range>;
    using uRange = std::remove_cvref_t<URange>
    if (std::ranges::empty(range)) {
        // ❌!
        // ... the below errors
        return uRange(std::forward<Range>(range));
    }
    /* perform some work with the
    iterators or similar */
    auto first = std::ranges::begin(range);
    auto last = std::ranges::end(range);
    if (*first == u'\0xEF') {
        // ...
        std::advance(first, 3);
        // ...
    }
    // ... algorithm finished,
    // return the "updated" range!

    // ❌!
    // ... but the below errors
    return uRange(std::move(first), std::move(last));
}

int main () {
    std::string_view meow_view = "나는 유리를 먹을 수 있어요. 그래도 아프지 않아요";
    // this line will error in C++17 and C++20, or in compatibility
    // libraries
    std::string_view sub_view
        = operate_on_and_return_updated_range(meow_view);
    return 0;
}
```

The current fix is to employ `std::ranges::subrange<I, S, K>` to return a generic subrange:

```
template <typename Range>
auto operate_on_and_return_updated_range (Range&& range) {
    using I = std::iterator_t<Range>;
    using S = std::sentinel_t<Range>;
    using Result = std::ranges::subrange<I, S>;
    if (std::ranges::empty(range)) {
        return Result(std::forward<Range>(range));
    }
    // perform some work with the
    // iterators or similar
    auto first = std::ranges::begin(range);
    auto last = std::ranges::end(range);
    if (*first == u'\0xEF') {
        // ...
        std::advance(first, 3);
        // ...
    }
    // ... algorithm finished,
    // return the "updated" range!

    // now it works!
    return Result(std::move(first), std::move(last));
}
```

```

}

int main () {
    std::string_view meow_view = "나는 유리를 먹을 수 있어요. 그래도 아프지 않아요";
    auto sub_view
        = operate_on_and_return_updated_range(meow_view);
    // decltype(sub_view) ==
    // std::ranges::subrange<std::string_view::iterator, std::string_view::iterator>
    // which is nowhere close to ideal.
    return 0;
}

```

This makes it work with any two pair of iterators, but quickly becomes undesirable from an interface point of view. If a user passes in a `quickcpplib::span<T, Extent>` or a `llvm::StringRef` that interface and information is entirely lost to the user of the above function. `std::ranges::subrange<Iterator, Sentinel, Kind>` does not -- and cannot/should not -- mimic the interface of the view it was created from other than what information comes from its iterators: it is the barebones idea of a pair-of-iterators/iterator-sentinel style of range. This is useful in the generic sense that if a library developer must work with iterators, they can always rely on creation of a `std::ranges::subrange` of the iterator and sentinel.

§ 2.1. Preservation of Properties

Unfortunately, always using `std::ranges::subrange` decreases usability for end users. Users who have, for example a `llvm::StringRef`, would prefer to have the same type after calling an algorithm. These types have functions and code that are purpose-made for `string_view`-like code: losing that intent means needing to reconstruct that from the ground up all over again. There is little reason why the original type needs to be discarded if it supports being put back together from its iterators.

The break-it-apart-and-then-generic-`subrange` approach also discards any range-specific storage optimizations and layout considerations, leaving us with the most bland kind of range similar to the "pair of iterators" model. Consider, for example, `llfio::span<T>`. The author of the Low Level File IO (LLFIO) library, Niall Douglass, has spent countless days submitting pull requests and change requests to MSVC, GCC, and Clang's standard libraries to ask them to change their structural layout to match things like the `iovec` of their platform or their asynchronous buffer span types. This optimization matters because it realizes the performance difference between having to individually transformed a container of `spans` to a more suitable type, or being able to simply `memcpy` the desired sequences of spans into the underlying asynchronous operations. If range operations were to be performed on the `llfio::span<T>` type, a `std::ranges::subrange` would hold, in most cases, two iterators rather than an iterator and a size. This completely eliminates the `memcpy` optimization. This is not the only place this happens, however: other types have different storages requirements that are not appropriately captured by their iterators in the most general sense, but may benefit from reconstruction and desired type information for the target range they should be constructed into.

§ 2.2. Compile-Time and Interoperability

Compilation time goes up as well in the current paradigm. Users must spawn a fresh `std::ranges::subrange<I, S, K>` for every different set of iterator/sentinel/kind triplet, or handle deeply nested templates in templates as the input types. This makes it impossible to compile interfaces as dynamic libraries without having to explicitly materialize or manually cajole a `std::ranges::subrange` into something more palatable for the regular world.

There is also a problem where there are a wide variety of ranges that could conceivably meet this criterion, but do not. The author of this paper was not the only one to see utility for this. [\[p1739r4\]](#) does much the same that this paper does, without the introduction of a concept to formalize the behavior it presents. In particular, it selects views which can realistically have their return types changed to match the input range and operations being performed (or a similarly powerful alternative) by asking whether they can be called with a function called `reconstruct` from a subrange of the iterators with the expressions acted upon.

- Ranges should be reconstructible from their iterators (or subrange of their iterators) where applicable;
- and, reconstructible ranges serve a useful purpose in generic algorithms, including not losing information and returning it in a much more cromulent and desirable form.

Therefore, this paper formalizes that concept and provides an **opt-in, user-overridable way** to return a related "reconstructed" type from an tag, an iterator, and a sentinel.

§ 3. Design

The design is given in 3 concepts added to the standard:

```
template <class It, class Sen = It>
concept iterator_reconstructible_range =
(std::is_class_v<It> || std::is_class_v<Sen> || std::is_enum_v<It> || std::is_enum_v<Sen>)
&& requires (It first, Sen last) {
    reconstruct(
        std::foward<It>(first),
        std::forward<Sen>(last)
    );
};

template <class R,
class It = ranges::iterator_t<R>,
class Sen = ranges::sentinel_t<R>>
concept reconstructible_range =
std::ranges::range<R>
&& requires (It first, Sen last) {
    reconstruct(
        in_place_type<remove_cvref_t<R>>,
        std::move(first),
        std::move(last)
    );
};

template <class R, class Tag = remove_cvref_t<R>,
class It = ranges::iterator_t<R>,
class Sen = ranges::sentinel_t<R>>
concept range_iterator_reconstructible_range =
std::ranges::range<R>
&& requires (R range, It first, Sen last) {
    reconstruct(
        in_place_type<Tag>,
        std::forward<R>(range),
        std::forward<It>(first),
        std::forward<Sen>(last)
    );
};

template <class R, class Tag = remove_cvref_t<R>>
concept range_reconstructible_range =
std::ranges::range<R>
&& requires (R range) {
    reconstruct(
        in_place_type<Tag>,
        std::forward<R>(range)
    );
};
```

It is the formalization that a range can be reconstructed from its necessary components. The concepts are a means of implementing the desired Customization Point Object (CPO) which will be called `std::ranges::reconstruct`. The 4 forms allow for 4 levels of construction, where each one defers to the next version by removing one layer of "information" until finally following back to the most basic form:

- the desired type to reconstruct ("tag"), the just the range;
- the desired type to reconstruct ("tag"), the range, the iterator, and the sentinel;

- the desired type to reconstruct ("tag"), the iterator, and the sentinel, after stripping out the range if the previous `reconstruct` did not find anything;
- the iterator and the sentinel, after dropping the tag if the previous `reconstruct` attempt did not work and making sure at least one of `std::is_class_v<iterator>`, `std::is_class_v<sentinel>`, `std::is_enum_v<iterator>`, or `std::is_enum_v<sentinel>` is `true`; and finally,
- `std::ranges::subrange<decltype(iterator), decltype(sentinel)>` if nothing else matches.

This allows a developer to put a range back together at various levels of fidelity from its parts and pieces, giving us the ability to propagate the input type's properties after modifying its iterators for some underlying work, algorithm or other effect. This concept is also the basis of the idea behind [\[p1739r4\]](#), which was accepted in a lesser form for C++20 with the intent of this paper following up on it.

§ 3.1. Range? Why not "Borrowed Range"?

Previously, we required that the ranges being reconstructed modeled `borrowed_range`, since that was an accurate indicator that the iterators and sentinel could potentially outlive the range itself. This was to prevent some issues in a previous design that used constructors. Since that is no longer based on constructors and with no chance of false positives, we no longer need to include that limitation. `borrowed_range` is also too restrictive, as there are some cases where the resulting range is non-borrowed, but could be successfully reconstructed from some of its pieces. The concept is opt-in now by way of declaring an ADL-found `reconstruct` function, any type can effectively decide for itself whether it could reconstruct properly. Note that this opens up usages for things that, if we had used `borrowed_range`, would not have been reconstructible. Consider the following expression:

```
std::vector<int> vec{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
std::span<int> some_span(vec);
some_span | views::transform(f) | views::take(5)
```

This particular case is reconstructible as `span<T>` is reconstructible. The original transformation view can be reconstructed here without the need to keep the `take_view` intermediate wrapped around the two previous transformations, as the transform can be ignored on the first five elements by advancing the given `some_span`'s `begin()` by 5 elements. A smart `reconstruct` call for `take_view` does this (and is present in [§ 6.3 Proposed Library Wording](#)).

§ 3.2. Multiple, Safe Reconstruction Points

Consider a hypothetical `c_string_view` type. It is impossible to, from normal `const char*` iterator pair ascertain the range is null-terminated and is thus a "C string". Therefore, a reconstruct for `c_string_view` would not return a `c_string_view`, but a normal `string_view`.

On the other hand, if there was a `c_string_sentinel` sentinel with a `const char*` iterator that was preserved through a series of actions and algorithms, that kind of information would allow us to return a `c_string_view` from a reconstruction operation.

The `reconstruct` extension point supports both of these scenarios:

```
class c_string_view {
    /* blah... */

    // reconstruct: with [const char*, const char*) iterators
    // no guarantee it's null terminated: decay to string_view
    friend constexpr std::string_view reconstruct (
        std::in_place_type_t<c_string_view>,
        const char* first, const char* last) noexcept {
        return std::string_view(first, last);
    }

    // reconstruct: with static [const char*, c_string_sentinel)
    // static guarantee by type: return a c_string_view
    friend constexpr c_string_view reconstruct (
```



```

        std::in_place_type_t<c_string_view>,
        const char* first, c_string_sentinel) noexcept {
            return c_string_view(first);
        }
};

```

This is a level of flexibility that is better than the Revision 0 constructor-based design, and can aid in providing better static guarantees while decaying gracefully in other situations.

§ 3.3. Opt-in?

Not all ranges can meet this requirement. Some ranges contain state which cannot be trivially propagated into the iterators, or state that cannot be reconstructed from the iterator/sentinel pair itself. However, most of the common ranges representing unbounded views, empty views, iterations viewing some section of non-owned storage, or similar can all be reconstructed from their iterator/iterator or iterator/sentinel pair.

For example `std::ranges::single_view` contains a [exposition *semiregular-box* template type](#) (`ranges.semi.wrap`) which holds a value to iterate over. It would not be possible to reconstruct the exact same range (e.g., iterators pointing to the exact same object) with the semi-regular wrapper. Ranges should have the ability to select this syntax without having it happen to their type implicitly, or as for more information.

This is why there are 4 levels to opt-in to support. If a user defines the 4-argument form which takes a `(tag, range, iterator, sentinel)` quadruplet, then they can use as much information may be available in the range, iterator, and sentinel to reproduce the original range. If they do not need that much information, they can opt-in to the 3-argument version which only takes the `(tag, iterator, sentinel)` triplet. If they want to only deal with the original range and the tag (as an optimization, to save on `reconstruct` potentially calling `ranges::begin(r)` and `ranges::end(r)` on the range and calling the `(tag, range, iterator, sentinel)` version), they can define `reconstruct` for just `(tag, range)`. And, finally, the most brief and sophisticated form is the `(iterator, sentinel)` form, which can reconstruct a range using information from only the iterator and/or sentinel.

If the range opts-in to nothing, then there will always be a default return: `std::ranges::subrange<Iterator, Sentinel>`. This is effectively what any algorithm today already returns (sometimes it uses a hand-made pair type, likely because some iterator-pair usages may require the size to be stored and neither the iterator nor the sentinel can provide the size).

§ 3.4. Applicability

There are many ranges to which this is applicable, but only a handful in the standard library need or satisfy this. If [\[p1391r2\]](#) and [\[p1394r2\]](#) are accepted (they were), then the two most important view types -- `std::span<T, Extent>` and `std::basic_string_view<Char, Traits>` -- will model the `_spirit_` of the concept (but lack the customization point to make it generic). `std::ranges::subrange<Iterator, Sentinel, Kind>` already fits this as well. By formalizing concepts in the standard, we can dependably and reliably assert that these properties continue to hold for these ranges. We intend to add a `constexpr friend reconstruct` function to all of the non-`ranges::subrange` types.

There are also upcoming ranges from [\[range-v3\]](#) and elsewhere that could model this concept:

- [\[p1255r4\]](#)'s `std::ranges::ref_maybe_view`;
- [\[p0009r9\]](#)'s `std::mdspan`;
- and, soon to be proposed by this author for the purposes of output range algorithms, [\[range-v3\]](#)'s `ranges::unbounded_view`.

And there are further range adaptor closure objects that could make use of this concept:

- `views::slice`, `views::take_exactly`, `views::drop_exactly` and `views::take_last` from [\[range-v3\]](#)

Note that these changes will greatly aid other algorithm writers who want to preserve the same input ranges. In the future, the standard may provide an `ranges::reconstruct(...)` algorithm for these types.

§ 3.5. Why The Overloads?

It has been questioned why there are four different potential extension points for `ranges::reconstruct`. Indeed, most individuals observe that all of the necessary information required is the range and (possibly) any outstanding iterators. Their version of this paper — termed the "Alternative Design" — looks like this:

```
template <typename Range, typename Iterator, typename Sentinel>
auto reconstruct(Range&& range, Iterator&& iterator, Sentinel&& sentinel);
```

It takes only the range, iterator, and sentinel. There are no tag types. There are no overloads which take: only an iterator and a sentinel; a tag, range, iterator and sentinel; a tag, iterator, and sentinel; or, a tag and a range.

While the user only has to override one in this paper's design, the design above provides **only** one customization point with no room for more or less information. However, many APIs which necessitate various forms of `reconstruct` due to various circumstances when either wrapping certain function / algorithms or dealing with non-generic code within generic code.

§ 3.5.1. Unrecoverable Ranges

Let us set up the need for the various reconstruction algorithms using a use case-by-contradiction. Consider, for a moment, an algorithm which takes a range and does some work (perhaps at a later point). A function is passed which is expected to take the given range and return a (any) range. It attempts to reconstruct the result at the end, if at all possible after some additional work is done on the resulting iterators:

```
template <typename Range, typename F>
auto perform_work_with(Range&& range, F&& f) {
    // things and stuff
    decltype(auto) result = std::forward<F>(f)(std::forward<Range>(range));
    auto result_first = std::ranges::begin(result);
    auto result_last = std::ranges::end(result);
    // more work is done with result_first and result_last
    auto pair_of_first_last
        = third_party_code(std::move(result_first), std::move(result_last));
    // other things and stuff
    // ...
    // return the combined range
    return std::ranges::reconstruct(std::move(result),
                                    std::move(pair_of_first_last.first),
                                    std::move(pair_of_first_last.second));
}
```

If we were to create a generic lambda with a universal reference parameter that worked something like:

```
auto f = [&output_buffer](auto&& r) {
    auto x = std::ranges::copy(std::forward<decltype(r)>(r), output_buffer.begin());
    return std::ranges::subrange(std::move(x.in), r.end());
};
```

It becomes vastly unclear whether or not the returned range is what we are supposed to be working with, or the range forwarded into the function. On one hand, we get more static information about what we should reconstruct by using the range we passed into `f` for `perform_work_with`. On the other hand, if we end up with a function that is not designed like `std::ranges::copy`, then working with the return value is necessary. That is, a normal user function, using `perform_work_with`, can take a value directly rather than a reference:

```
auto f2 = [&output_buffer](move_only_input_range r) { // by-value: uh oh !!
    auto x = std::ranges::copy(std::move(r), output_buffer.begin());
    return std::ranges::subrange(std::move(x.in), std::move(r.end()));
};
```

The above `f2` is entirely consistent with the ["What is a View?" paper](#), which argues that specific ranges should (still) be taken by-value, while generic range algorithms need to be taken by (forwarding) reference. This means that when we do `std::forward<Range>(range)` inside of `perform_work_with`, we do not get to call `rbegin()` on the original `range` anymore. The only way to keep working is with `result`.

Effectively, we have lost the original range and its type information by the time we return from this function. And, this, it turns out, becomes pretty important in Generic-with-a-capital-G code.

§ 3.5.1.1. Negotiating

Okay, so in general we can't always work with the original range anymore. This is, in fact, something that happens pretty often. For example, in a range-based text library `ztd.text`, [the third-party encoding for SHIFT-JIS takes and returns `std::span`] (https://github.com/soasis/text/blob/81db6a2b0cf5bdf3ab7fbd8b2ba2744fa6ce636d/examples/shift_jis/include/shift_jis.hpp#L56). However, more often than not, users of the algorithms with these encoding objects work with and provide `std::string` or `std::string_view`-like types. If we were to directly return what the results of the third-party encoding gave us, we would be forced to always give back to the user a `std::span` (or, in the worst case, `std::subrange<T*>`), not a `std::string_view` or `std::string`-capable type.

Under the Alternative Design, this results in a problem for both `f` and `f2`: it will always be called with the parameter types like so:

```
reconstruct(std::ranges::subrange<iterator, sentinel>, iterator, sentinel);
```

This presents significant problems for reconstruction that requires all 3 of the `range`, `first`, and `last` to reconstruct: how do you write a reconstruction point for this? If neither the `begin` nor `end` of the range have namespace-relation to the range type (e.g., the iterators are 2 pointers, [as in `llvm::StringRef` from LLVM] (https://llvm.org/doxygen/classllvm_1_1StringRef.html#a20d37563688a61a452fb26e317e37308) or [`TArrayView` from Unreal Engine](<https://docs.unrealengine.com/4.26/en-US/API/Runtime/Core/Containers/TArrayView/begin/>)), ADL will not pick up if you use `subrange` as the range type. For example, the following reconstruction point — even though we technically have all the necessary information to rebuild an e.g. Unreal Engine `TArrayView<T>` — will not be called if we used it `f` and `perform_work_with`:

```
template <typename T>
TArrayView<T> reconstruct(TArrayView<T> original, T* first, T* last);
```

We can try to write another reconstruction point for subranges, using the following:

```
template <typename T>
TArrayView<T> reconstruct(subrange<T*> original, T* first, T* last);
```

But this definition would collide with a standard-provided reconstruction point:

```
namespace std::ranges {
    template <typename It, typename Sen>
        subrange<It, Sen> reconstruct(subrange<It, Sen> original, It first, Sen last);
}
```

which the wording below provides, regardless of the design, to meet the pre-existing uses for reconstruct-like behavior in `drop_view` and `take_view`.

At this point, we've ran ourselves in a circle: all we wanted to do was provide the best possible reconstruction point, and now — due to the limitations of the Alternative Design API — we have potentially created an ADL reconstruction point that could conflict with the standard library. So we have 2 choices:

- do not use the Alternative Design; or,

- leave it so we always just return whatever the `iterator/sentinel` from `f` might be, as a `subrange`, after we do any additional work.

At this point, we're still interested in recovering some kind of range after performing our work based on the iterators.

§ 3.5.1.2. Current Design: Lossless Information

The Current Design does not suffer from this issue. Even if the original range is lost (moved from, consumed by an intermediate algorithm, or otherwise), the Current Design still allows reconstruction from differently-typed ranges and iterators. This, at the very least, strongly motivates the tag type that is passed to 3 of the reconstruction customization points found today.

```
// Current design:
// can always be found using ADL with the tag type
// always safe (tag type serves as strong type to match against)
friend constexpr
template <typename T>
TArrayView<T>
reconstruct(std::in_place_type_t<TArrayView<T>>, T*, T*);
```

Note the benefits of this approach:

- only the iterator/sentinel and a tag are needed to reconstruct (the tag can be taken from either the return value or from the input to the algorithm, whichever the algorithm designer thinks suits their users best);
- information can be recovered further up the chain using purely type information, rather than values directly; and,
- there is far less of a chance for ambiguities or lookup confusion.

This may seem like is a very simplified/conjured scenario, but it has real-world implications. In the library `ztd.text`, input and output ranges are taken and worked on. Eventually, these algorithms [call user-defined functions](#) through `encode_one` or `decode_one` functions. Ranges are returned in structures. These are not required to return the same - or even directly related - range from their function calls. They could even return `std::ranges::subrange<some_iterator>` instead of the original range, which may explicitly discard information necessary for reconstruction.

Again, as [\[P2415\]](#) indicates, an end-user can take a range by-value since not all encoding objects need to be fully generic. In these algorithms and as pointed out above, that may be a destructive operation, and from there we have to work with the returned range. The returned range may have a severely reduced amount of type information, and therefore it is in our best interests as a library author to attempt to reconstruct based on the input, and not exactly what the third-party intermediary works in.

The Current Design means that we get type-safe reconstruction even in the face of such user-defined or third-party algorithms. It works in the general sense, and allows us to capture user intents and semantics better than the Alternative Design. It also does not set users up to attempt to write reconstructions for ranges that they do not fundamentally own, like `std::ranges::subrange`.

§ 3.5.2. Legacy Code

As much as the authors personally hate legacy code, there are at least 20 years of `[iterator, iterator)` code floating out in the world. Many times, we lack a range or range abstraction in general. Depriving the vast majority of this code from being able to properly reconstruct their types, even if they have strongly-typed iterators that are not basic pointer types, is very hostile to the current ecosystem.

For example, [CharacterIterator from ICU](#) works using an (abstract) base class design which a user can override to provide iteration semantics over their own storage types. Some information may not be perfectly recoverable after transmuting it to their format (for example, using the pre-defined class for handling an array of characters). There are many cases where third party libraries and code will require us to fundamentally transmute a pair of iterators or a range to something that cannot be recovered easily, even in generic code.

§ 3.5.3. Minimum Viable Customization

In proprietary codebases, we have been able to reconstruct `string_views`, `spans`, `c_string_views`, and various kinds of `rope_views` and other kinds of views purely from their iterators. Most of the views in the standard can be reconstructed using their iterator and sentinel: while more ranges will come in the future which may require more and additional information, it is extraordinarily clear that the majority of ranges both users (and the standard) will be using will boil down to using `(iterator, sentinel)` or `(tag, iterator, sentinel)` information for reconstruction purposes. Most users simply do not need the range information, and so basing the design on this facet is unnecessarily hostile to generic code that has been trafficking in iterators and wants to steadily evolve to returning more information, potentially without API breaks (for example, going from returning a `Boost.Range`-style `subrange` and instead reconstructing to something better).

Whether we like it or not, `(iterator, sentinel)` is still the fundamental unit of work for algorithms and ranges, NOT a single `[range)/[cursor)` type. Disregarding two decades of such code and always requiring a range to work is not in the C++ Committee's best interests for proliferation of range-based ideas and code.

It is also important to note that requiring the Alternative Design's `(range, iterator, sentinel)` customization point means that a user may have to write an overload for every single `(range, iterator, sentinel)` triplet. To save on all of this writing, either the first parameter will become templated (which can result in overload problems for ranges which may have many accidental potential matches), or each range needs to be carefully crafted to take the proper range with an iterator and sentinel (e.g. when the iterator and sentinel may be primitive types like `T*`).

§ 4. Deployment Experience

This change was motivated by Hannes Hauswedell's [\[p1739r4\]](#) and became very important for the ranges work done with text encoding. There is a C++17 implementation [at the ztd.text repository here](#), which is an implementation for the interface needed for [\[p1629r1\]](#). It is meant to solve the deployment issues with P1739's merging into the Standard.

§ 5. Impact

As a feature that is opt-in thanks to the `ranges::reconstruct` Customization Point Object design, no currently created range or previously made range is affected.

Furthermore, this is a new and separate set of concepts. It is not to be added to the base `Range` concept, or added to any other concept. It is to be applied separately to the types which can reasonably support it for the benefit of algorithms and code which can enhance the quality of their implementation.

Finally, Hannes Hauswedell's [\[p1739r4\]](#) with the explicit intention to mark certain ranges as reconstructible by hardcoding their behavior into the standard and come back with an opt-in fix during the C++23 cycle. This paper completes that promise.s

§ 6. Proposed Changes

The following wording is relative to the latest C++ Draft paper.

§ 6.1. Feature Test Macro

This paper results in a concept to help guide the further development of standard ranges and simplify their usages in generic contexts. There is one proposed feature test macro, `__cpp_lib_reconstructible_range`, which is to be input into the standard and then explicitly updated every time a `reconstruct` from a is added to reflect the new wording. We hope that by putting this in the standard early, most incoming ranges will be checked for compatibility with `pair_reconstructible_range` and `reconstructible_range`.

§ 6.2. Intent

The intent of this wording is to provide greater generic coding guarantees and optimizations by allowing for a class of ranges and views that model the new exposition-only definitions of a reconstructible range:

- add a new feature test macro for reconstructible ranges to cover constructor changes;
- add a new customization point object for `ranges::reconstruct`;
- add the customization point to many different ranges that can be reconstructed;
- and, add two new concepts to [\[range.req\]](#).

§ 6.3. Proposed Library Wording

§ 6.3.1. Add a feature test macro `__cpp_lib_reconstructible_range`.

§ 6.3.2. Insert into §24.2 Header `<ranges>` Synopsis [\[ranges.syn\]](#) a new customization point object in the inline namespace:

```
namespace std::ranges {  
  inline namespace unspecified {  
    ...  
  
    inline constexpr unspecified reconstruct = unspecified;  
  
    ...  
  }  
}
```

§ 6.3.3. Insert into §24.4.2 Ranges [\[range.range\]](#)'s after paragraph 7, one additional paragraph:

⁸The concepts `iterator_reconstructible_range`, `reconstructible_range`, `range_iterator_reconstructible_range`, and `range_reconstructible_range` concepts describe the requirements on ranges that are efficiently constructible from certain iterator and sentinel types, alongside possible additional information from the range itself.

```
template <class It, class Sen = It>
    concept iterator_reconstructible_range =
        (std::is_class_v<It> || std::is_class_v<Sen>
         || std::is_enum_v<It> || std::is_enum_v<Sen>).
    && requires (It first, Sen last) {
        reconstruct(
            std::forward<It>(first),
            std::forward<Sen>(last)
        ).i;
    };
```

```
template <class Tag,
    class It = ranges::iterator_t<R>,
    class Sen = ranges::sentinel_t<R>>
    concept reconstructible_range =
        std::ranges::range<R>
    && requires (It first, Sen last) {
        reconstruct(
            in_place_type<Tag>,
            std::move(first),
            std::move(last)
        ).i;
    };
```

```
template <class R, class Tag = remove_cvref_t<R>,
    class It = ranges::iterator_t<R>,
    class Sen = ranges::sentinel_t<R>>
    concept range_iterator_reconstructible_range =
        std::ranges::range<R>
    && requires (R range, It first, Sen last) {
        reconstruct(
            in_place_type<Tag>,
            std::forward<R>(range),
            std::forward<It>(first),
            std::forward<Sen>(last)
        ).i;
    };
```

```
template <class R, class Tag = remove_cvref_t<R>>
    concept range_reconstructible_range =
        std::ranges::range<R>
    && requires (R range) {
        reconstruct(
            in_place_type<Tag>,
            std::forward<R>(range)
        ).i;
    };
```

⁹Let r be a range with type R , T be some type, i be an iterator of type I and s be a sentinel of type S .

^{9.1} — Let `it Sen re_range` be the result of `ranges::reconstruct(i, s)` if I and S satisfy `ranges::iterator_reconstructible_range`, r models `ranges::iterator_reconstructible_range` if

— `i == ranges::begin(it Sen re_range)` is true, and

— `s == ranges::end(it Sen re_range)` is true.

9.2 — Let `re_range` be the result of `ranges::reconstruct(in_place_type<remove_cvref_t<T>>, i, s)` if `remove_cvref_t<T>`, `I`, and `S` satisfy `ranges::iterator_reconstructible_range`. `r` models `ranges::reconstructible_range` if

— `i == ranges::begin(re_range)` is true, and

— `s == ranges::end(re_range)` is true.

9.3 — Let `range_it_re_range` be the result of `ranges::reconstruct(in_place_type<remove_cvref_t<T>>, r, i, s)`, if `R`, `remove_cvref_t<T>`, `I`, and `S` satisfy `ranges::range_iterator_reconstructible_range`. Then `range_it_re_range` models `ranges::range_iterator_reconstructible_range` if

— `i == ranges::begin(range_it_re_range)` is true, and

— `s == ranges::end(range_it_re_range)` is true.

9.4 — Let `range_re_range` be the result of `ranges::reconstruct(in_place_type<remove_cvref_t<T>>, r)`, if `in_place_type<remove_cvref_t<T>>` and `R` satisfy `ranges::range_reconstructible_range`. Then `range_re_range` models `ranges::range_reconstructible_range` if

— `ranges::begin(r) == ranges::begin(range_re_range)` is true, and

— `ranges::end(r) == ranges::end(range_re_range)` is true.

[Note: If an iterator and a sentinel is passed in that does not come directly from a `ranges::begin` or `ranges::end` expression, then the range of the reconstructed range matches the given iterator and sentinel so long as it is appropriately related. — end Note]

- § 6.3.4. Insert a new sub-clause "§24.3.13 **ranges::reconstruct** [range.prim.recons]", after "§24.3.12 **ranges::cdata** [range.prim.cdata]":

24.3.12 **ranges::reconstruct**

[range.prim.recons]

¹The name `reconstruct` denotes a [customization point object](#).

²The expression `ranges::reconstruct(I, S)` for some sub-expressions `I` and `S` is expression-equivalent to:

(2.1) `reconstruct(std::forward<decltype(I)>(I), std::forward<decltype(S)>(S))` if either `remove_cvref_t<decltype(I)>` or `remove_cvref_t<decltype(S)>` are class or enumeration types, it is a valid expression, and both `decltype(I)` and `decltype(S)` model `iterator_reconstructible_range`.

(2.2) Otherwise, `ranges::subrange<remove_cvref_t<decltype(I)>, remove_cvref_t<decltype(S)>>(std::forward<decltype(I)>(I), std::forward<decltype(S)>(S)))` if it is a valid expression.

(2.3) Otherwise, `ranges::reconstruct(I, S)` is ill-formed.

³The expression `ranges::reconstruct(in_place_type<R>, I, S)` for some type `R` and some sub-expressions `I` and `S` is expression-equivalent to:

(2.1) `reconstruct(in_place_type<R>, std::forward<decltype(I)>(I), std::forward<decltype(S)>(S))` if it is a valid expression and `R`, `decltype(I)`, and `decltype(S)` model `reconstructible_range`.

(2.2) Otherwise, `ranges::reconstruct(std::forward<decltype(I)>(I), std::forward<decltype(S)>(S))`.

⁴The expression `ranges::reconstruct(in_place_type<R>, SR, I, S)` for some type `R`, and some sub-expressions `SR`, `I`, and `S` is expression-equivalent to:

(2.1) `reconstruct(in_place_type<R>, std::forward<decltype(SR)>(SR), std::forward<decltype(I)>(I), std::forward<decltype(S)>(S))` if it is a valid expression and `decltype(SR)`, `R`, `decltype(I)`, and `decltype(S)` model `range_reconstructible_range`.

(2.2) Otherwise, `ranges::reconstruct(in_place_type<R>, std::forward<decltype(I)>(I), std::forward<decltype(S)>(S))`.

- § 6.3.5. Add to "§21.4.3.1 **General**" [string.view.template.general] a **friend** function ADL extension point for **basic_string_view**:

```
// [string.view.reconstruct]
friend constexpr basic_string_view
reconstruct(std::in_place_type_t<basic_string_view>,
            const_iterator first, const_iterator last) noexcept;
```

- § 6.3.6. Add a new subclause "§21.4. **Range Reconstruction**" [string.view.reconstruct] in §21.4:

21.4. **Range Reconstruction**

[string.view.reconstruct]

```
friend constexpr basic_string_view
reconstruct(in_place_type_t<basic_string_view>,
            const_iterator first, const_iterator last) noexcept;
```

¹Returns: `basic_string_view(std::move(first), std::move(last))`.

§ 6.3.7. Add to "§22.7.3.1 **Overview**" [[span.overview](#)] a **friend** function ADL extension point for **span**:

```
// [span.reconstruct].  
template <class It, class End>  
friend constexpr auto  
reconstruct(in_place_type_t<span>, It first, End last) noexcept;
```

§ 6.3.8. Add a new subclause "§22.7.3. **Range Reconstruction**" [[span.reconstruct](#)] in §22.7.3:

22.7.3. **Range Reconstruction**

[[span.reconstruct](#)]

```
template <class It, class End>  
friend constexpr auto  
reconstruct(in_place_type_t<span>, It first, End last) noexcept;
```

¹ Constraints: Let U be `remove_reference_t<iter_reference_t<It>>`.

(1.1) — is_convertible_v<U(*)[], element_type(*)[]> is true. [*Note: The intent is to allow only qualification conversions of the iterator reference type to element type. — end note*]

(1.2) — It satisfies contiguous_iterator.

(1.3) — End satisfies sized_sentinel_for<It>.

(1.4) — is_convertible_v<End, size_t> is false.

² Let MaybeExtent be an implementation-defined constant expression of type size_t. If it is not equal to dynamic_extent, then MaybeExtent shall have a value equal to distance(first, last).

³ Returns: `span<ElementType, MaybeExtent>(std::move(first), std::move(last)).`

⁴ Remarks: the return type may be promoted to a `span` with a static extent if the implementation is capable of deriving such information from the given iterator and sentinel.

[*Note 1:* some iterator types are implementation-defined, and may carry additional information which allow determining a non-dynamic_extent value for MaybeExtent. — end note]

§ 6.3.9. Add to "§24.5.4.1 **General**" [[range.subrange.general](#)] a **friend** function ADL extension point for **subrange**:

```
friend constexpr subrange  
reconstruct(in_place_type_t<subrange>, I first, S last) noexcept(see-below);  
  
friend constexpr auto  
reconstruct(in_place_type_t<subrange>, I first, S last) noexcept(see-below);
```

§ 6.3.10. Add a new subclause "§24.5.4. **Range Reconstruction** " [range.subrange.reconstruct] in §24.5.4:

22.7.3. **Range Reconstruction**

[range.subrange.reconstruct]

```
friend constexpr subrange  
reconstruct(in_place_type_t<subrange>, I first, S last) noexcept(see-below);
```

¹ Returns: subrange(std::move(first), std::move(last)).

² Throws: Anything from evaluating the *Returns*. Otherwise, nothing.

```
friend constexpr auto  
reconstruct(in_place_type_t<subrange>, I first, I last) noexcept(see-below);
```

³ Returns: subrange<I>(std::move(first), std::move(last)).

⁴ Throws: Anything from evaluating the *Returns*. Otherwise, nothing.

§ 6.3.11. Add to "§24.6.4.2 Class template **iota_view**" [range.iota] a **friend** function ADL extension point for **iota_view**:

```
template <typename S>  
friend constexpr iota_view  
reconstruct(iterator first, S last) noexcept(see-below);
```

§ 6.3.12. Add one new paragraph to "§24.6.4.2 Class template **iota_view**" [range.iota]:

```
template <typename S>  
friend constexpr auto  
reconstruct(iterator first, S last) noexcept(see-below);
```

  ¹ Constraints: S is *iterator* or S is *sentinel*.

  ² Effects: Equivalent to:

```
if constexpr (same_as<iterator, S>) {  
    return iota_view<decltype(*std::move(first)), decltype(*std::move(last))>(  
        std::move(first), *std::move(last)  
    ).  
};  
else {  
    return iota_view(std::move(first), std::move(last)).  
};
```

  ³ Throws: Anything from evaluating the *Effects*. Otherwise, nothing.

- § 6.3.13. Add to "§24.6.2.2 Class template `empty_view`" [[range.empty.view](#)] a **friend** function ADL extension point for `empty_view`:

```
friend constexpr empty_view
reconstruct(in_place_type_t<empty_view>, T*, T*) noexcept {
    return empty_view{};
}

friend constexpr empty_view
reconstruct(in_place_type_t<empty_view>, nullptr_t, T*) noexcept {
    return empty_view{};
}

friend constexpr empty_view
reconstruct(in_place_type_t<empty_view>, T*, nullptr_t) noexcept {
    return empty_view{};
}

friend constexpr empty_view
reconstruct(in_place_type_t<empty_view>, nullptr_t, nullptr_t) noexcept {
    return empty_view{};
}
```

- § 6.3.14. Add to "§24.7.6.2 Class template `transform_view`" [[range.transform.view](#)] a **friend** function ADL extension point for `transform_view`:

```
template <bool iterator_condition, bool sentinel_condition>
friend constexpr auto
reconstruct(iterator<iterator_condition> first,
            sentinel<sentinel_condition> last) noexcept(see-below);
```

- § 6.3.15. Add new paragraphs to "§24.6.4.2 Class template `transform_view`" [[range.transform](#)]:

```
template <bool iterator_condition, bool sentinel_condition>
friend constexpr auto
reconstruct(iterator<iterator_condition> first,
            sentinel<sentinel_condition> last) noexcept(see-below);
```

◆◆¹ *Constraints:* V models `reconstructible_range` and F models `copy_constructible`.

◆◆² *Returns:* `transform_view(ranges::reconstruct(std::in_place_type<V>, std::move(first).current, std::move(last).end), *first.parent->fun_)`.

◆◆³ *Throws:* Anything from evaluating the *Returns*. Otherwise, nothing.

```
template <bool iterator_condition, bool sentinel_condition>
friend constexpr auto
reconstruct(in_place_type_t<transform_view>, transform_view original,
            iterator<iterator_condition> first,
            sentinel<sentinel_condition> last) noexcept(see-below);
```

◆◆⁴ *Constraints:* V models `reconstructible_range`.

◆◆⁵ *Returns:* `transform_view(ranges::reconstruct(std::in_place_type<V>, std::move(first).current, std::move(last).end), std::move(*original.fun_))`.

◆◆⁶ *Throws:* Anything from evaluating the *Returns*. Otherwise, nothing.

§ 6.3.16. Modify "§24.7.7.1 Overview " [\[range.take.overview\]](#) for `views::take`'s paragraph 2:

² The name `views::take` denotes a range adaptor object (`[range.adaptor.object]`). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::take(E, F)` is ill-formed. Otherwise, the expression `views::take(E, F)` is expression-equivalent to:

~~(2.1) — If `T` is a specialization of `ranges::empty_view` (`[range.empty.view]`), then `((void) F, decay-copy(E))`.~~

~~(2.2) — Otherwise, if `T` models `random_access_range` and `sized_range` and is~~

~~(2.2.1) — a specialization of `span` (`[views.span]`) where `T::extent == dynamic_extent`;~~

~~(2.2.2) — a specialization of `basic_string_view` (`[string.view]`);~~

~~(2.2.3) — a specialization of `ranges::iota_view` (`[range.iota.view]`), or~~

~~(2.2.4) — a specialization of `ranges::subrange` (`[range.subrange]`);~~

~~then `T{ranges::begin(E), ranges::begin(E) + min<D>(ranges::size(E), F)}`, except that `E` is evaluated only once.~~

~~(2.3) — Otherwise, `ranges::take_view{E, F}`.~~

(2.1) — If `T` is a specialization of `ranges::empty_view` (`[range.empty.view]`), then `((void) F, decay-copy(E))`.

(2.2) — Otherwise, if `T` models both `random_access_range` and `sized_range` and:

(2.2.1) — `T`, `remove_cvref_t<T>`, `ranges::iterator_t<T>`, and `ranges::iterator_t<T>` model `range_iterator_reconstructible_range`, or

(2.2.2) — `T`, `ranges::iterator_t<T>`, and `ranges::iterator_t<T>` model `reconstructible_range`, or

(2.2.3) — `ranges::iterator_t<T>` models `iterator_reconstructible_range`.

then `ranges::reconstruct(in_place_type<T>, E, ranges::begin(E), ranges::begin(E) + min<D>(ranges::size(E), F))`, except that `E` is evaluated only once.

(2.3) — Otherwise, `ranges::take_view{E, F}`.

§ 6.3.17. Modify "§24.7.9.1 Overview" [[range.drop.overview](#)] for `views::drop`'s paragraph 2:

² The name `views::drop` denotes a range adaptor object (`[range.adaptor.object]`). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::drop(E, F)` is ill-formed. Otherwise, the expression `views::drop(E, F)` is expression-equivalent to:

~~(2.1) — If `T` is a specialization of `ranges::empty_view` (`[range.empty.view]`), then `((void) F, decay-copy(E))`.~~

~~(2.2) — Otherwise, if `T` models `random_access_range` and `sized_range` and is~~

~~(2.2.1) — a specialization of `span` (`[views.span]`) where `T::extent == dynamic_extent`;~~

~~(2.2.2) — a specialization of `basic_string_view` (`[string.view]`);~~

~~(2.2.3) — a specialization of `ranges::iota_view` (`[range.iota.view]`), or~~

~~(2.2.4) — a specialization of `ranges::subrange` (`[range.subrange]`);~~

~~then `T{ranges::begin(E) + min<D>(ranges::size(E), F), ranges::end(E)}`, except that `E` is evaluated only once.~~

~~(2.3) — Otherwise, `ranges::drop_view{E, F}`.~~

(2.1) — If `T` is a specialization of `ranges::empty_view` (`[range.empty.view]`), then `((void) F, decay-copy(E))`.

(2.2) — Otherwise, if `T` models both `random_access_range` and `sized_range`, and `T`:

(2.2.1) — `T.remove_cvref_t<T>`, `ranges::iterator_t<T>`, and `ranges::sentinel_t<T>` model `range_iterator_reconstructible_range`, or

(2.2.2) — `T`, `ranges::iterator_t<T>`, and `ranges::sentinel_t<T>` model `reconstructible_range`, or

(2.2.3) — `ranges::iterator_t<T>` and `ranges::sentinel_t<T>` model `iterator_reconstructible_range`.

then `ranges::reconstruct(in_place_type<T>, E, ranges::begin(E) + min<D>(ranges::size(E), F), ranges::end(E))`, except that `E` is evaluated only once.

(2.3) — Otherwise, `ranges::drop_view{E, F}`.

§ 7. Acknowledgements

Thanks to Corentin Jabot, Christopher DiBella, and Hannes Hauswedell for pointing me to [\[p1035r7\]](#) and [\[p1739r4\]](#) to review both papers and combine some of their ideas in here. Thanks to Eric Niebler for prompting me to think of the generic, scalable solution to this problem rather than working on one-off fixes for individuals views.

Thank you to Oktal, Anointed of ADL, Blessed Among Us, and Morwenn, the ever-watching Code Guardian for suggesting improvements to the current concept form.

§ References

§ Informative References

[ICU-CHARITERATOR]

Unicode Consortium; International Components for Unicode. [CharacterIterator: ICU Documentation](#). March 22nd, 2021. URL: <https://unicode-org.github.io/icu/userguide/strings/characteriterator.html>

[N4835]

Richard Smith. [Working Draft, Standard for Programming Language C++](#). 8 October 2019. URL: <https://wg21.link/n4835>

[P0009R9]

H. Carter Edwards, Bryce Adelstein Lelbach, Daniel Sunderland, D. S. Hollman, Christian Trott, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Mark Hoemmen. [mdspan: A Non-Ownning Multidimensional Array Reference](https://wg21.link/p0009r9). 20 January 2019. URL: <https://wg21.link/p0009r9>

[P1035R7]

Christopher Di Bella, Casey Carter, Corentin Jabot. [Input Range Adaptors](https://wg21.link/p1035r7). 2 August 2019. URL: <https://wg21.link/p1035r7>

[P1255R4]

Steve Downey. [A view of 0 or 1 elements: view::maybe](https://wg21.link/p1255r4). 17 June 2019. URL: <https://wg21.link/p1255r4>

[P1391R2]

Corentin Jabot. [Range constructor for std::string_view](https://wg21.link/p1391r2). 17 June 2019. URL: <https://wg21.link/p1391r2>

[P1394R2]

Corentin Jabot, Casey Carter. [Range constructor for std::span](https://wg21.link/p1394r2). 17 June 2019. URL: <https://wg21.link/p1394r2>

[P1629R1]

JeanHeyd Meneide. [Transcoding the world - Standard Text Encoding](https://wg21.link/p1629r1). 2 March 2020. URL: <https://wg21.link/p1629r1>

[P1739R4]

Hannes Hauswedell. [Avoid template bloat for safe_ranges in combination with 'subrange-y' view adaptors](https://wg21.link/p1739r4). 1 March 2020. URL: <https://wg21.link/p1739r4>

[P2415]

Barry Revzin; Tim Song. [What is a View?](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2415r0.html). July 13th, 2021. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2415r0.html>

[RANGE-V3]

Eric Niebler; Casey Carter. [range-v3](https://github.com/ericniebler/range-v3). June 11th, 2019. URL: <https://github.com/ericniebler/range-v3>